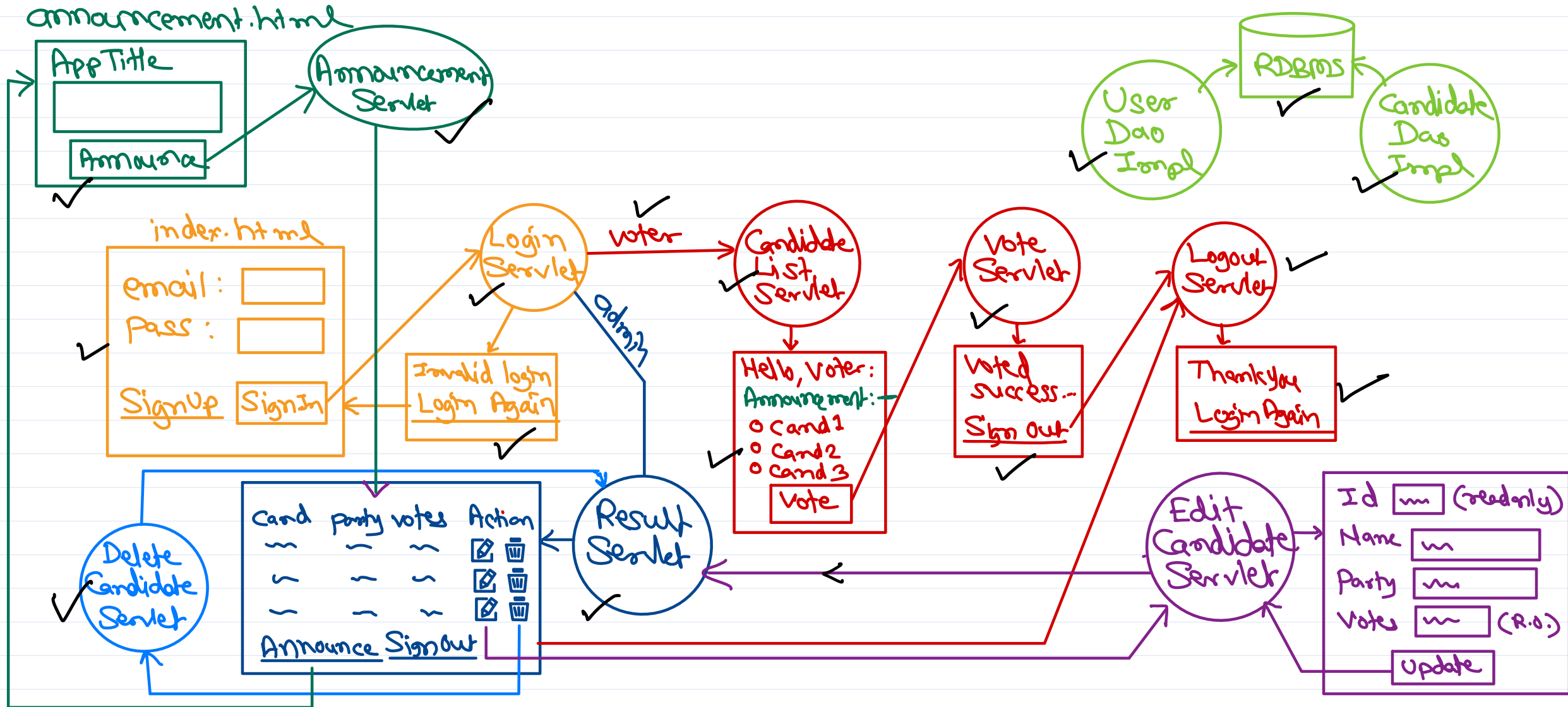
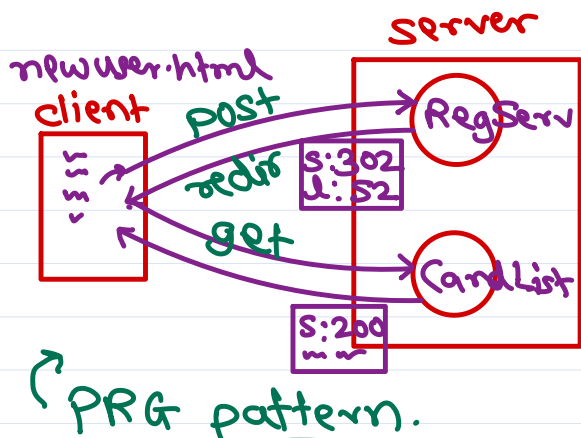
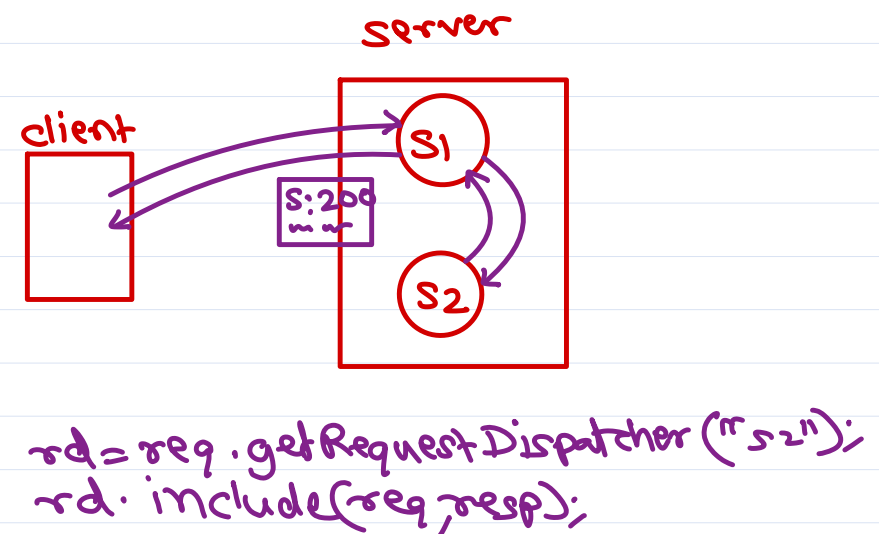
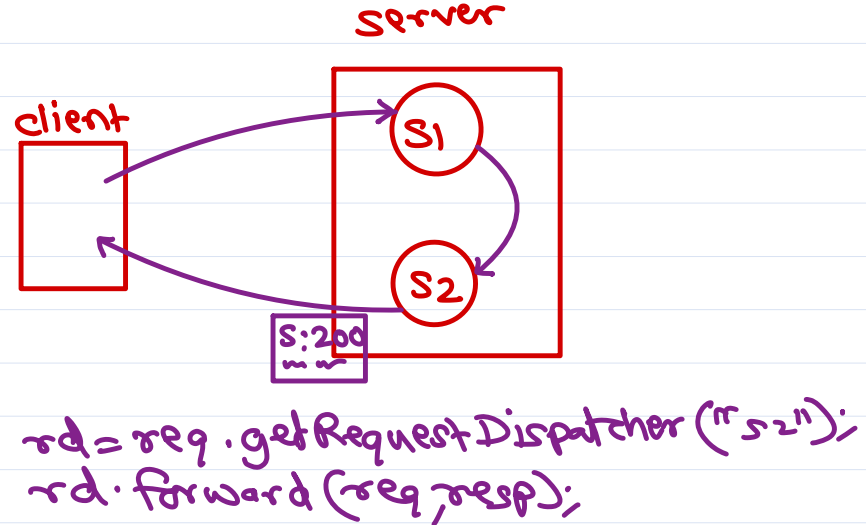
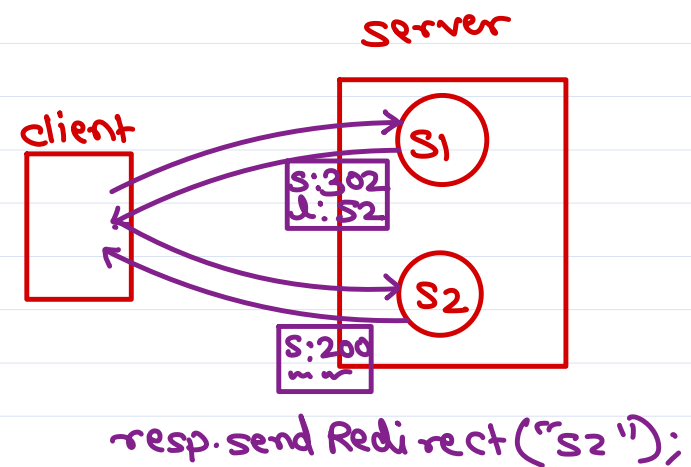


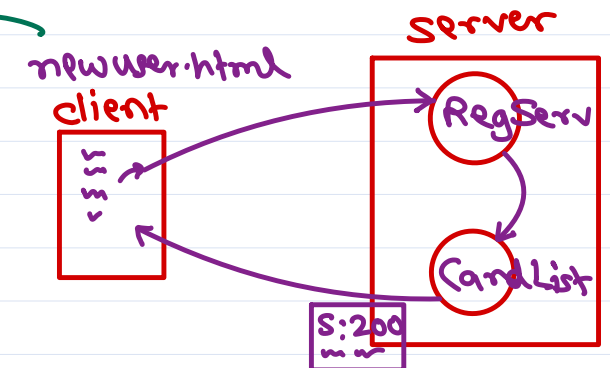
Election Management



Inter-Servlet Communication



when client refresh, last req (ie. post) is replayed, due to which data is sent twice & may be added or updated in db twice.



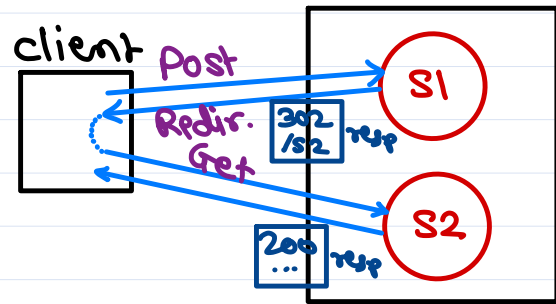
if client refresh, only last req (GET) is replayed & data fetched again. Earlier data not posted twice.



Servlet Communication/Navigation

HTTP Redirection

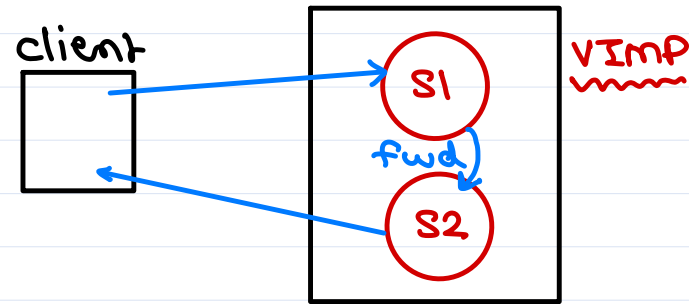
```
resp.sendRedirect("url");
```



- ① execution as shown in diag. Two diff req from browser - slower.
- ② url in browser is changed
- ③ can navigate to any page in or outside the web appln.
- ④ Useful after Post req, so that same data is not posted again upon refresh.
Redirection → PRG pattern.

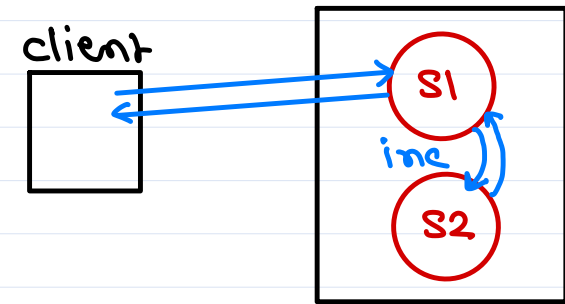
forward() ← Request Dispatcher → include()

```
rd = req.getRequestDispatcher("url");  
rd.forward(req, resp);
```



- ① same req is fwded to next page - faster.
- ② url in browser is unchanged - browser not aware of navigation.
- ③ can navigate to any page inside the web appln.
- ④ final resp generated by next page.

```
rd.include(req, resp);
```



- ① same req is fwded to next page - faster.
- ② url in browser is unchanged - browser not aware of navigation.
- ③ can navigate to any page inside the web appln.
- ④ final resp is sent by first page.



State Management

* maintaining state/info about client is called as "state mgmt".

* Client side state mgmt

- Cookie
- QueryString
- Hidden fields
- ~~HTML Storage~~

- ✓ Need less server resources.
- ✗ Visible to client
- ✗ Client can tamper

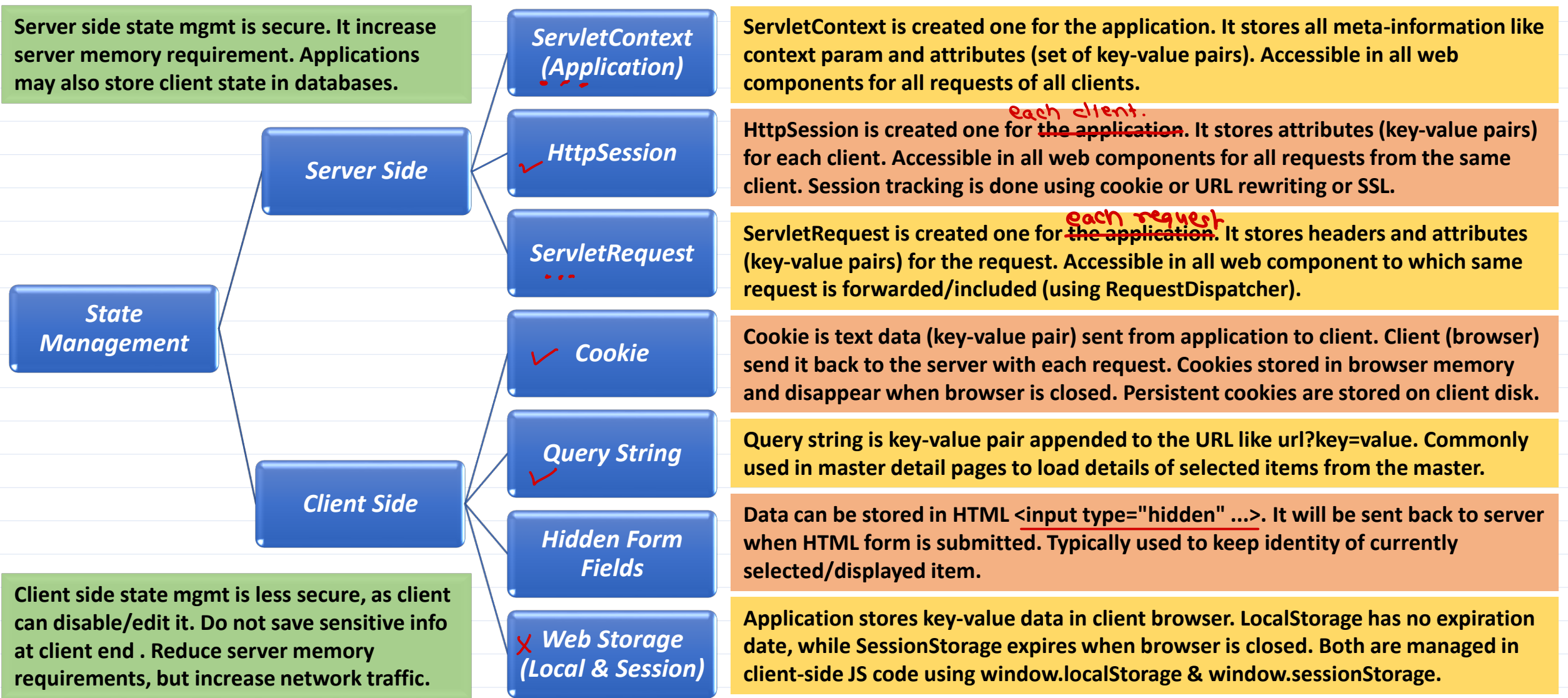
* Server side state mgmt

- Session
- Request
- Application a.k.a. ServletContext

- ✓ Secure (Not accessible to client)
- ✗ Need more server resources.



Java Web Applications – State management – at a glance



Cookie

Cookie → text key value pair

- stored on client side

- temp cookie - browser memory.
- persistent cookie - client disk until expiry time.

- max size = 4 KB

- created by server and sent to client in a response.

- afterwards cookie is sent back to server with each request by the client.

- cookies can be seen and modified by the client.

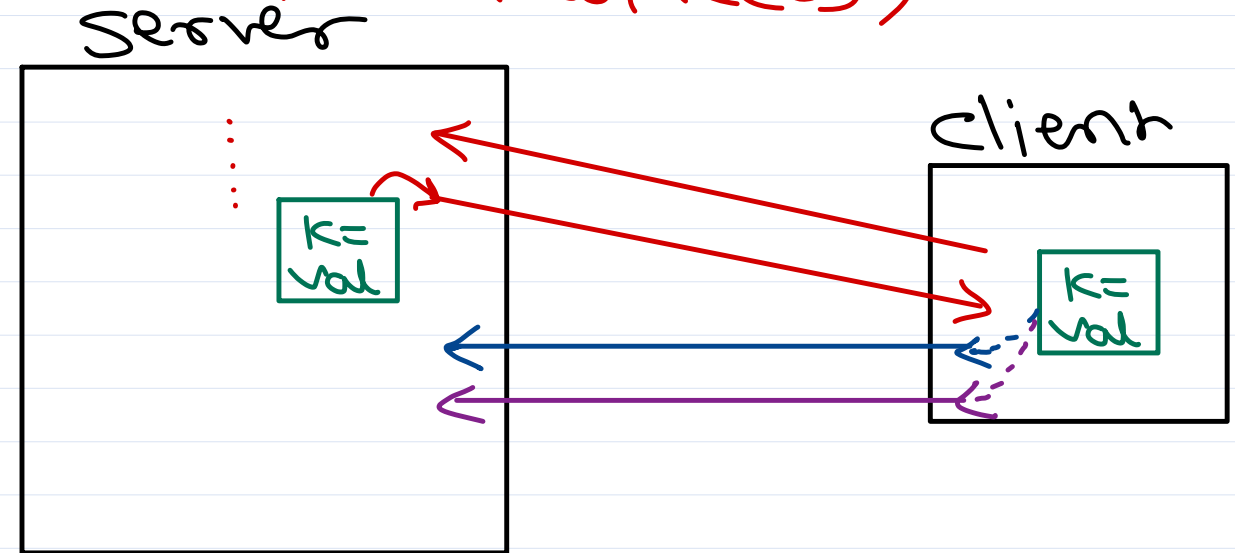
- Cookies can be disabled in browser.

`c.setMaxAge(-1);` → cookie become temp.

`c.setMaxAge(0);` → cookie delete

* Send cookie to client:

```
Cookie c = new Cookie("k", "v");  
c.setMaxAge(seconds); // ← persistent cookie  
resp.addCookie(c);
```



* receive cookie from client:

```
Cookie[] arr = req.getCookies();  
for (Cookie c : arr) {  
    if (c.getName().equals("k")) {  
        v = c.getValue();  
        break;  
    }  
}
```



Session

Http Session is key-value map stored on server side for each user.

To create/access session:

```
Session = req.getSession();
```

To store key-value in session:

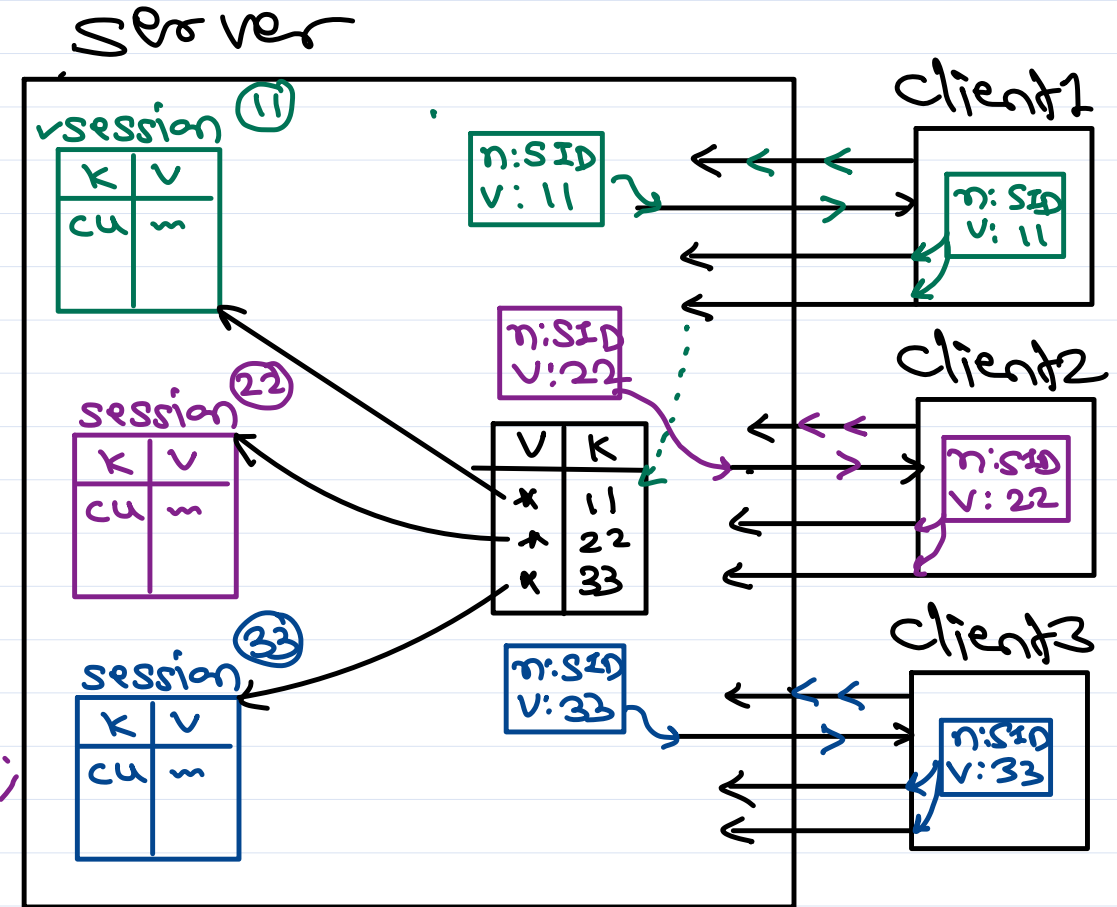
```
session.setAttribute("key", value);
```

To retrieve key-value from session:

```
value = session.getAttribute("key");
```

To destroy the session:

```
session.invalidate();
```



Session tracking
Using Cookie (JSSESSIONID).



Session

```
session = req.getSession();
```

- ① check if session id cookie is present in current request. If available, find the Session object corresponding to that id and return it.
- ② if session id cookie doesn't exist, create a new session and save it with new session id (in server internal mem). Create a new session id cookie and send it to client with next response.

Session tracking modes:

web.xml

```
<session-config>  
  <tracking-mode> COOKIE </tracking-mode>  
  or  
  </session-config> URL
```

Session tracking by URL rewriting

http://lh:8080/app/url
(after re-writing)
http://lh:8080/app/url;jsessionid=xxxxx

Programmer need to rewrite each url in app (form action, a href, resp.sendRedirect, ...) using

- ① reurl = resp.encodeURL(url);
- OR
② reurl = resp.encodeRedirectURL(url);

QueryString

when req next page, the page url can be appended with additional key-value pairs.

It is called as **query string**. ↴

`http://lh:8080/app/page?key1=val1&key2=val2`

This data is accessible in that page as `req.parameters`.

```
String val1 = req.getParameter("key1");
```

```
String val2 = req.getParameter("key2");
```



Req parameters vs Req attributes

- ① received from client.
- ② usually data from form controls or query string.
- ③ always string.

④ to access:
String v = req.getParameter("-");
or
String[] v = req.getParameterValues("-");

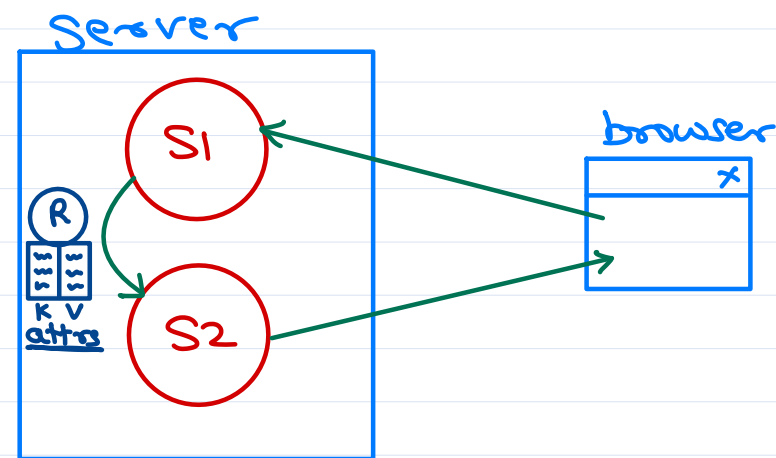
* both are destroyed when req processing is completed i.e. response is generated

- ① sent from servlet1 to servlet2
- ② any data added by servlet1.
- ③ any java Object
- ④ servlet 1:
req.setAttribute("k", val);
servlet 2:
val = req.getAttribute("k");
- ⑤ Used only while forwarding/
including request.



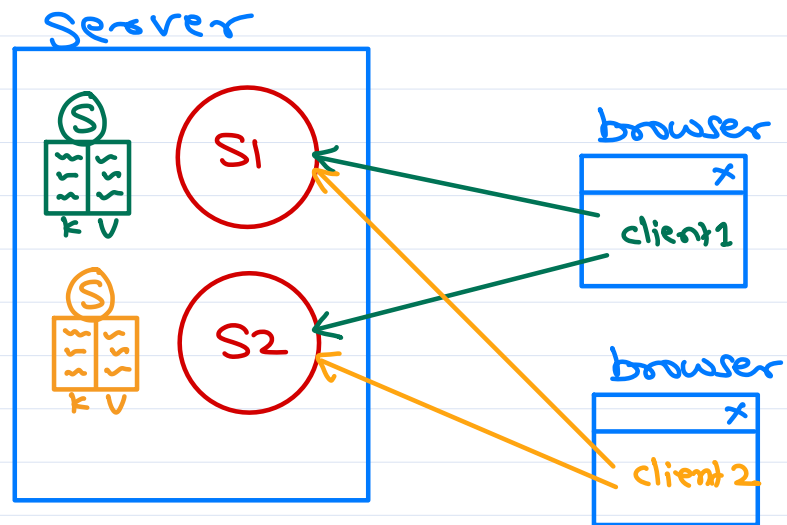
Attributes - Scopes

Request attributes



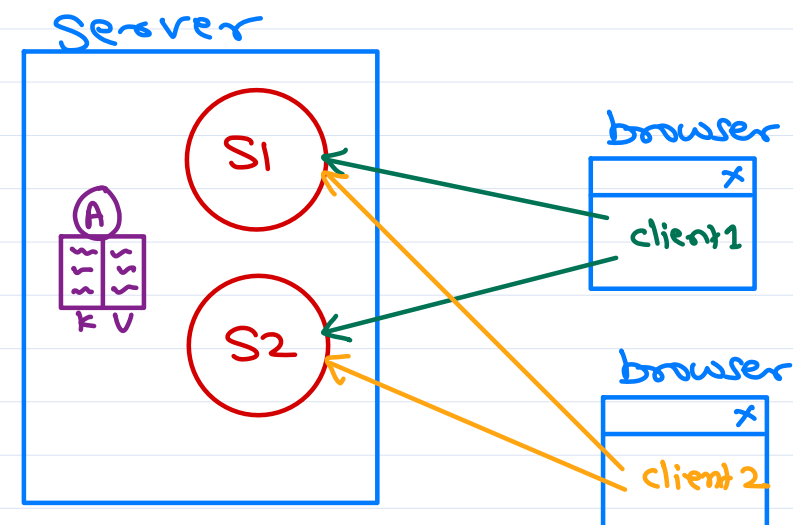
Req attributes will live only for current request-being forwarded or included.

Session Attributes



Session attributes are accessible across all requests for current user.
Separate session for each user.

Servlet Context attributes



ServletContext - application object - one for whole appn.
ServletContext attributes are accessible across all requests for all users.

"req" obj is arg to doGet(), doPost(), ... methods.

To get current user session:
`HttpSession session = req.getSession();`

To get appn/ServletContext:
`ServletContext appn = req.getServletContext();`

